

lab1实验报告

任务介绍

本实验是编译器设计的正式开端，后续实验都在本实验生成的AST树上进行。本实验的内容包括：

- 编写词法分析文件lexer.l
- 编写语法分析文件parser.y
- 编写用于构建和遍历打印AST的ast.cpp,ast.hpp

以及用于枚举转换运算符和标识符的common.hpp和入口文件main.cpp。可以在main.cpp开启调试以及添加图形化方法的调用。

实现的功能

本实验中实现了以下功能：

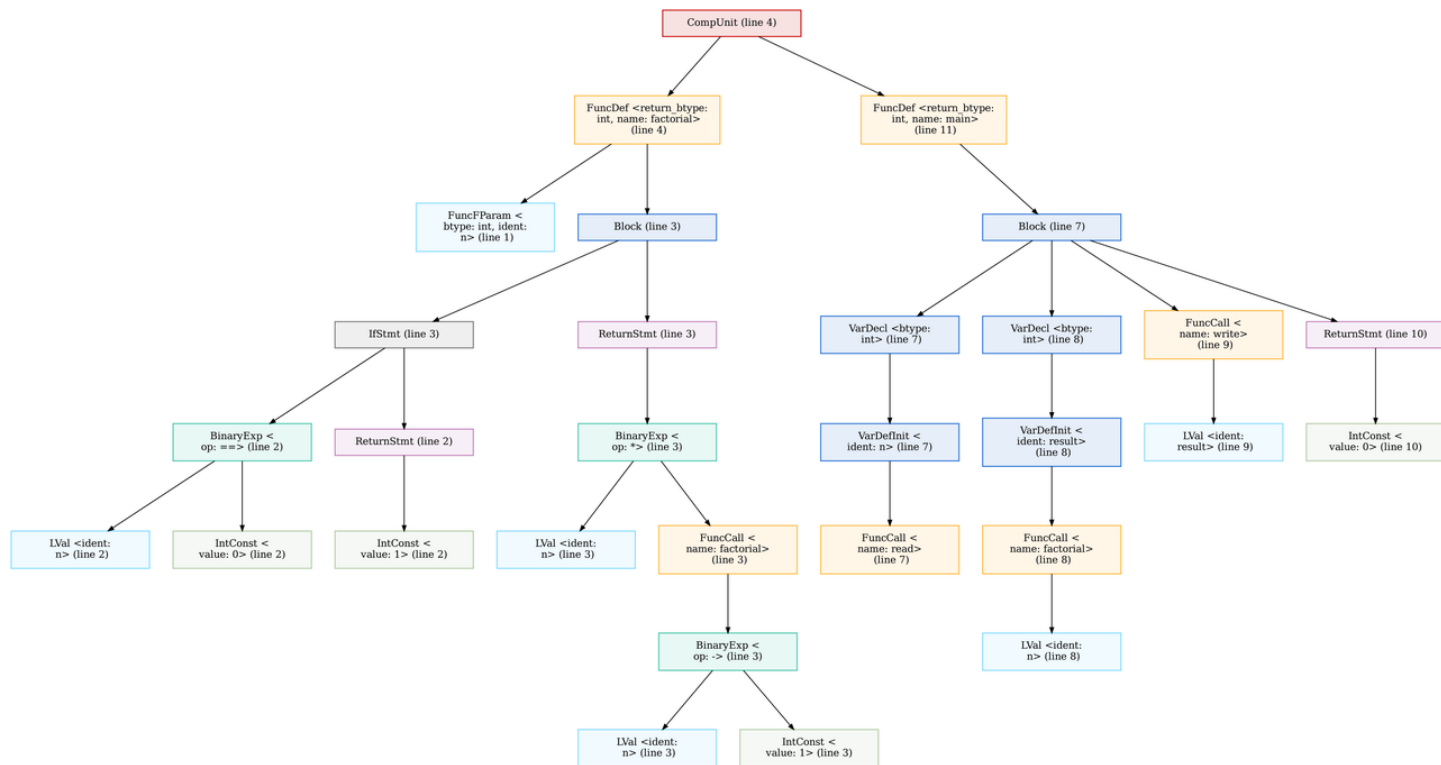
1. 对读取的程序文件使用flex进行词法分析，生成token流并记录每个token的行号。
2. 对接收的token流使用bison根据SysY语法规则进行语法分析,同时构建AST节点。
3. 在解析过程中，为每条语法规则创建相应的AST节点，同时通过每个节点类根据其语法结构存储其子节点，以及语法规则的动作部分构建节点并建立关系，来建立节点之间的父子关系，最终生成完整的AST，根节点为CompUnit。
4. 实现了除了模版自带的树状打印以外的图形化：遍历树，根据节点类型应用不同颜色（使用NodeColor类），然后统一设置节点形状、大小和文本格式，并使用HTML标签实现文本自动换行，最后生成Graphviz格式的dot文件。

整体效果如下：

示例输入：factorial.sy

```
1  int factorial(int n) {
2      if (n == 0) return 1;
3      return n * factorial(n - 1);
4  }
5
6  int main() {
7      int n = read();
8      int result = factorial(n);
9      write(result);
10     return 0;
11 }
```

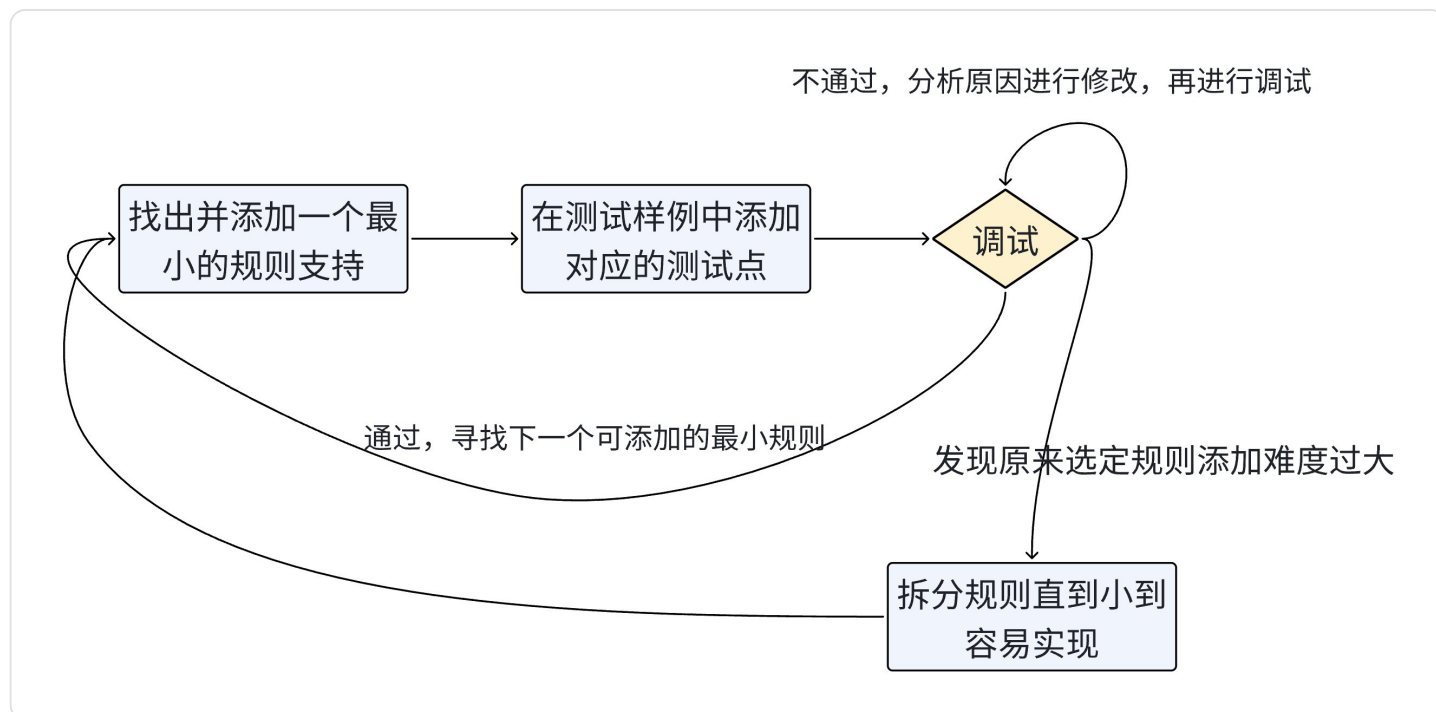
示例输出：



实现中的亮点

最小化迭代式更新开发

本实验基于给定的模版进行**最小化迭代式更新**，每次只添加语言中最小的可测试功能单元，确保每步都能稳定工作，基本流程如下：



实现步骤如下：

1. 设计测试样例覆盖模版已实现的规则，通过测试
2. 添加*/%!的识别和-*,/,%,!表达式的识别
3. 添加全局变量声明的支持
4. 添加变量初始化
5. 添加函数参数列表定义
6. 支持 void 类型函数
7. 支持条件语句与Cond表达式
8. 支持while循环
9. 支持while-if嵌套
10. 支持数组声明
11. 支持通过数组下标访问
12. 支持数组作为函数参数
13. 支持括号表达式
14. 支持空语句
15. 支持八进制和十六进制整数常量
16. 添加图形化方法
17. 完成图形化参数调节

我认为这种方法特别适合编译器这类复杂系统的开发，避免了一次引入过多功能导致的调试困难。

智能的AST可视化系统

实现了一个具有高度定制性的AST可视化系统，包含的子功能包括：

- **语义化颜色编码：**不同类型的节点使用不同颜色，使AST结构一目了然。
- **自适应文本处理：**使用HTML标签实现长标签自动换行，确保节点内容完整显示。
- **优化的图形布局：**通过调整节点间距和层级布局，生成美观且信息密度适中的图形。
- **节点信息整合：**每个节点集成显示源代码位置信息，便于代码追踪。

考虑到AST树是后面实验进行的基础，我认为将可视化做得好一点是很有必要的，这会为之后的设计和调试工作带来不少便利。

AI工具与代码借鉴使用声明

本实验本人没有借鉴任何除统一提供的模版之外的代码。

本报告没有使用AI工具编写。

本实验的代码部分由本人进行构思和调试以及设计性编写，而结构重复的节点定义和图形化方法的细节调整由AI（github copilot）生成。

建议和想法

我觉得本次实验的指导书和模版非常的全面和友好，有效地平滑了实验的难度曲线，感谢助教为此做出的贡献！